IdempotentWallet: Study Guide in Simple English (IT3133)

Revised for the final scope: Idempotency & Exactly-Once(ish) only · Oct 17, 2026 · @Pehesara

1. The story in one page

In one sentence: We built a digital wallet where pressing 'Send' five times still moves the money only once. We prove it by breaking the system on purpose in front of the examiners.

What the lecturer is testing. The module is Programming Distributed Components. The big idea: when programs talk over a network, things fail halfway, and the same request can arrive twice. The cure (Distributed Design, slide 4) is: send messages at-least-once, and make the receiver idempotent using a stable operation ID. Our project is a working proof of that sentence.

Our scope. The lecturer asked our group for one design pattern only: Idempotency & Exactly-Once(ish). So the project is one service, one database and one broker. We removed the saga, the second service, the resilience tools (circuit breaker, bulkhead) and tracing, because they belong to other slides.

The story in five acts

1. **The problem.** A customer taps 'Send LKR 500'. The Wi-Fi drops. The customer taps again. Did the money move once or twice? The server cannot tell a lost request from a lost reply.
2. **The rule.** Never overwrite a balance. Only add a new line to a permanent statement (an 'event'). This is event sourcing. It gives a full audit trail, so we can prove how many times money moved.
3. **The guard.** Every money request carries an `Idempotency-Key`. The database remembers key -> result. A repeat gets the saved receipt, not a second debit.
4. **The pipeline.** Events leave the database through an outbox, travel through RabbitMQ, and the receiver skips duplicates using an inbox.

5. **The proof.** We cause failures on purpose (lost replies, duplicate messages, a poison message, 50 identical requests at once) and show that money is never created or lost by a retry.

Why 'Exactly-Once(ish)'? Perfect exactly-once delivery over a network is impossible. What we build is **effectively-once**: a message may arrive many times, but its effect happens once. The 'ish' is the honest word for that gap. Saying it confidently in the viva earns marks.

What is in the project

Piece	What it is	Port
wallet-service	Wallets, ledger, idempotent API, outbox relay, notification consumer, chaos switches	8080
PostgreSQL	One database called wallet	5432
RabbitMQ	The message broker (UI at localhost:15672)	5672
Tests	Unit test, end-to-end tests, a k6 load script	src/test, k6/
Demo files	requests.http and the Postman collection	project folders

In numbers: 1 service, 1 database, 1 broker, 2 idempotency layers, 6 demo scenarios.

Where the code has a weakness, this guide says so (section 11). Examiners reward students who name their own weak spots.

2. Foundations: the ideas under everything

2.1 Partial failure

A distributed system is several programs on different machines sending messages to each other. Its main problem is **partial failure**: one part can fail while the others keep working. Analogy: texting a friend. 'Delivered' does not mean 'read', and silence does not mean 'no'.

2.2 The lost-reply problem (the heart of the project)

A client sends 'top up LKR 500' and hears nothing. There are three possible worlds, and the client cannot tell which one it is in:

- The request never arrived. Money did not move.
- The server crashed before saving. Money did not move.
- The server saved the top-up, but the reply was lost. Money moved.

The only safe choice is to retry. So retries are required, and duplicates are unavoidable. The whole project exists to make a duplicate harmless. (This is like the Two Generals problem: two parties on an unreliable link can never be fully sure the other got the last message.)

2.3 Delivery guarantees

Guarantee	Meaning	Weakness
At-most-once	Delivered 0 or 1 times. Send once, never retry.	Messages can be lost
At-least-once	Delivered 1 or more times. Retry until acknowledged.	Messages can be duplicated
Exactly-once	Delivered exactly 1 time.	Impossible over a network in general

Our recipe: use at-least-once (nothing is lost), then make the receiver idempotent (duplicates change nothing). Result: **effectively-once**.

2.4 Idempotency

An operation is idempotent if doing it many times has the same effect as doing it once. Analogy: a lift button. Pressing it five times still calls one lift. A light switch labelled 'set to ON' is idempotent. One labelled 'toggle' is not.

Operation	Idempotent?	Why
Set balance to 500	Yes	Same end state however often you run it
Add 500 to balance	No	Every run changes the result
HTTP GET, PUT, DELETE	Yes (by specification)	They describe a state, not an increment

Operation	Idempotent?	Why
HTTP POST (create a top-up)	No by default	Each call creates something new

Money movements are 'add 500' operations that arrive by POST, so they are not idempotent naturally. We **make** them idempotent: give each intention a stable ID (the `Idempotency-Key`) and store key -> result. The key belongs to the intention ('this one top-up'), not to one network attempt, so every retry carries the same key.

2.5 CAP and eventual consistency

CAP theorem: during a network partition you must choose between Consistency (everyone sees the latest write) and Availability (everyone gets an answer). Partition tolerance is not optional. Our choice: ledger writes favour consistency per wallet (a conflicting write is rejected and retried). Between the database and the broker (notifications) the system is eventually consistent: the consumer catches up a moment later.

2.6 Ordering without clocks

The lectures cover Lamport and vector clocks. We do not need them: each wallet has its own counter `seq` (1, 2, 3...). One order per wallet is enough. Choosing the simplest tool that works is a design skill worth mentioning.

2.7 Where each idea comes from in the slides

Idea	Lecture	Used in the project as
Partial failure, retries	01 Introduction, slide 24	Lost-reply story, retry with jitter
CAP, eventual consistency	03 Distributed Programming	Consistency choices above
Idempotency and Exactly-Once(ish)	04 Distributed Design, slides 4-5	The two idempotency layers
Transactional outbox	04 Distributed Design, slide 3	Outbox table and relay
Message middleware	05 Middleware	RabbitMQ

Idea	Lecture	Used in the project as
Race conditions, deadlocks	06 Parallel Processing	Optimistic concurrency, parallel tests
Stubs, mocks, boundaries	07 Component Testing	Tests (section 8)

3. Architecture

```
Client -- POST /wallets/{id}/topups + Idempotency-Key --> wallet-service
:8080 --> PostgreSQL 'wallet'
                          ONE transaction: key row + ledger event + outbox
row
                                        |
   outbox relay (every 1 s) --> RabbitMQ --> notification consumer (inbox
dedupe)
```

The client calls the REST API. The wallet-service writes to PostgreSQL and, later, announces events through RabbitMQ.

Why only one service? The pattern we were asked for is about repeated requests and repeated messages. One service with a database and a broker already has both boundaries: client to API, and database to broker to consumer.

Design rules

- **Idempotent at every boundary.** Every place a request or message can repeat has a duplicate guard.
- **One transaction for the whole effect.** Key, ledger event and outbox row commit together.
- **Messages for anything leaving the database.** The broker decouples in time: if the consumer is down, messages wait.
- **One writer per wallet.** Only wallet-service writes a wallet's ledger, so ordering is simple.

Flow A: a top-up

1. Client sends `POST /wallets/{id}/topups` with an `Idempotency-Key`.
2. `IdempotentExecutor` opens a database transaction and claims the key.

3. `LedgerService` adds a Credited event, updates the balance row, and writes an outbox row, all in the same transaction.
4. The transaction commits and the receipt is saved against the key. Client gets 201.
5. `OutboxRelay` (runs every second) publishes the outbox row to RabbitMQ with a stable message ID.
6. `NotificationConsumer` saves the message ID in the inbox and writes a notification in one transaction. A repeated message is skipped.

Flow B: the lost reply (what the examiners will see)

1. Client sends a top-up. The server commits it. The reply is lost (we switch on `dropResponse`), so the client sees 503.
2. The client retries with the **same** key.
3. The insert into `idempotency_keys` finds the key already there. The server returns the stored receipt with `Idempotent-Replayed: true`.
4. The balance went up once, not twice.

Who does what

Component	Job	Idea shown
WalletController	Public REST API (wallets, top-ups, withdrawals, balance, events, replay)	Idempotent API design
IdempotentExecutor	Claims the key, runs the effect, saves the receipt, in one transaction	Idempotency layer 1
RequestHasher	Makes a SHA-256 fingerprint of scope and amount	Detects key reuse with another payload
ConflictRetry	Retries a write that lost a race, with random delay	Optimistic concurrency, jitter
IdempotencyCleanup	Deletes keys older than 24 hours	TTL
LedgerService	Adds events, updates balance, writes outbox row, replays history	Event sourcing

Component	Job	Idea shown
OutboxRelay	Publishes outbox rows to RabbitMQ	Transactional outbox
NotificationConsumer	Applies an event once using the inbox	Idempotency layer 2
ApiExceptionHandler	Maps errors to 422, 409, 404, 400	Clear error answers
ChaosController, ChaosState	Switches that break things on purpose	Fault injection
Flyway V1__init.sql	Creates every table, versioned	Database migration
docker-compose.yml	Starts PostgreSQL and RabbitMQ with one command	Repeatable environment

4. Data layer: PostgreSQL and the tables

The database is not just storage. It is the **referee**. It can say 'no' atomically even when many requests collide.

4.1 Transactions and ACID

A transaction is a group of changes that succeed or fail together. ACID gives four promises:

Letter	Promise	Analogy	Where we use it
Atomicity	All changes happen, or none	Bank transfer: debit and credit both, or neither	Key row + ledger event + outbox row commit together
Consistency	Rules are never broken	A form that refuses a blank required field	amount_minor > 0, UNIQUE (wallet_id, seq)
Isolation	Concurrent transactions do not see each other's half-done work	Two cashiers with separate notepads	Duplicate requests are lined up by the unique index

Letter	Promise	Analogy	Where we use it
Durability	Once committed, it survives a crash	Ink, not pencil	A committed event is never lost

Isolation level. PostgreSQL defaults to READ COMMITTED: each statement sees only data committed before it began. When two identical requests insert the same key, the second INSERT waits for the first to finish. If the first commits, the second sees the row and replays it. If the first rolls back, the second becomes the owner.

4.2 Constraints: rules the database enforces

- **PRIMARY KEY / UNIQUE:** no two rows may share the value. This is our doorman.
- **CHECK:** a rule on values, for example `amount_minor > 0`.
- **FOREIGN KEY:** a row must point to a real parent (an event needs a real wallet).
- **ON CONFLICT DO NOTHING:** 'try to insert; if the key exists, do nothing and tell me zero rows changed'. This is how we safely ask the doorman.

4.3 Why money is a bigint, never a decimal

We store minor units (LKR 10.00 is stored as 1000). Floating-point numbers cannot store values like 0.1 exactly, so sums drift (0.1 + 0.2 gives 0.30000000000000004). Whole numbers never drift.

4.4 Every table and why it exists

Table	Purpose	Key detail
wallets	One row per wallet	Parent for events and balance
ledger_events	Source of truth: every credit and debit, append-only	UNIQUE (wallet_id, seq) plus a trigger that forbids update and delete
wallet_balance	Read model: current balance and last seq	Updated only WHERE last_seq = ? (optimistic concurrency)
idempotency_keys	Layer 1: key, request hash, saved response, expiry	Primary key (scope, idem_key)

Table	Purpose	Key detail
outbox	Messages waiting to be published	Same transaction as the data; status PENDING, PUBLISHED or DEAD
inbox	Layer 2: message IDs already processed	Primary key (consumer, message_id)
notification_log	Proof of the consumer's side effect	UNIQUE (message_id): one row even if delivered twice

4.5 Three SQL ideas to know by heart

1. Append-only ledger, enforced by the database. Even a buggy program cannot rewrite history:

```
CREATE TRIGGER ledger_events_immutable
BEFORE UPDATE OR DELETE ON ledger_events
FOR EACH ROW EXECUTE FUNCTION forbid_ledger_change();
```

2. The doorman. The unique index plus ON CONFLICT DO NOTHING decides who owns a key:

```
INSERT INTO idempotency_keys (scope, idem_key, request_hash, status,
expires_at)
VALUES (?, ?, ?, 'IN_PROGRESS', now() + interval '24 hours')
ON CONFLICT (scope, idem_key) DO NOTHING
```

3. Several relays without double publishing. SKIP LOCKED makes a second worker skip rows another worker already holds:

```
SELECT id, event_id, topic, payload FROM outbox
WHERE status = 'PENDING' ORDER BY id LIMIT 50
FOR UPDATE SKIP LOCKED
```

4.6 Flyway migrations (and the error we really hit)

Flyway runs numbered SQL scripts (V1__init.sql, V2__...) in order and remembers which ones ran, with a checksum of each. Analogy: version control for the database structure. **Rule: never edit a script that already ran. Add a new version instead.**

We broke this rule ourselves during setup. The app refused to start with: 'Migration checksum mismatch for migration version 1'. The database had the old V1, the file on disk had the new V1. Because the data was only demo data, the fix was `docker compose down -v` (delete the database volume) and `docker compose up -d`, so Flyway applied the current V1 to a clean database. In a real system you would add V2 instead. Telling this story in the viva shows you understand why migrations are versioned.

5. Event sourcing and the ledger

5.1 Event sourcing

Instead of storing the current state (balance = 600), store the history of facts that made it (credited 1000, debited 400). The balance is calculated by adding up the history. Analogy: a bank statement versus a whiteboard. A whiteboard shows only today's number, and wiping it destroys the past. A statement is a list of lines that are never erased.

Events are facts in the past tense: Credited, Debited. They are never edited. A mistake is fixed by adding a new compensating event, like a bank reversal.

Benefit	Cost
Full audit trail: who moved what and when	Reading the balance means adding up events (we avoid this with a stored balance)
Replay: rebuild the state at any time	The stored balance is a second copy that must stay in step
Fits messaging: each event can be published	Changing an event's shape later needs care
Bugs are recoverable: fix the code, replay the events	More concepts for the team to learn

Why it matters for idempotency: the ledger is our proof. After a storm of duplicates, we count the events: exactly one.

5.2 CQRS

Command Query Responsibility Segregation: use one model for changing data and a different model for reading it. Analogy: the cashier's ledger book (careful, authoritative) versus the notice board in the lobby (quick to read, built from the ledger).

- **Write side:** `ledger_events`, the source of truth.
- **Read side:** `wallet_balance`, a projection built from the events.

Honest simplification: both live in one database and the projection updates in the same transaction, so it is always up to date. Real CQRS usually updates the read side later (eventual consistency). Say this in the viva.

5.3 How LedgerService.append works

1. Read the current balance and `last_seq`.
2. Check the rules. A debit bigger than the balance throws InsufficientFundsException.
3. Compute `seq = last_seq + 1` and insert the event. The unique `(wallet_id, seq)` stops a second writer from taking the same number.
4. Update the balance row only if `last_seq` is still what we read.
5. Insert the outbox row so the event is published later.

```
long seq = current.lastSeq() + 1;
jdbc.update(INSERT INTO ledger_events ..., ...);
int updated = jdbc.update(
  UPDATE wallet_balance SET balance_minor = ?, last_seq = ?
  WHERE wallet_id = ? AND last_seq = ?, ...);
if (updated == 0)
  throw new DomainErrors.OptimisticConflictException();
```

`Propagation.MANDATORY` means the method refuses to run unless a transaction is already open. So nobody can write an event outside the idempotent transaction.

5.4 Optimistic concurrency

Instead of locking a record so others must wait (pessimistic), let everyone proceed and detect a clash at the moment of writing (optimistic). The loser retries on fresh data. Analogy: two people editing a shared document. On saving, the second is told 'someone changed it, reload and redo'.

Worked example: the classic overdraft race. A wallet holds 100. Two withdrawals of 80 arrive at the same instant.

1. Both read balance 100 and `last_seq` 1. Both think 80 is fine.
2. Both try to insert event `seq = 2`. The unique index lets one in. The other fails with a duplicate-key error.

3. The loser's transaction rolls back. `ConflictRetry` waits a few random milliseconds and runs it again.
4. On the retry it reads balance 20, so the withdrawal now fails with INSUFFICIENT_FUNDS.

Without the unique constraint, both would succeed and the wallet would be overdrawn. This is the race condition from the Parallel Processing lecture, solved without holding locks, so there is no circular wait and no deadlock.

5.5 Replay: proof the read model is disposable

`GET /wallets/{id}/balance/replay` recomputes the balance only from events and returns both numbers with `consistent: true` if they match. **Not implemented:** snapshots (saving the balance every N events so replay stays fast). Mention as future work.

6. Idempotency in depth (the marks section)

This is the section the lecturer cares about most. Know it well enough to draw it on a whiteboard.

6.1 Two layers, two different duplicate sources

Layer	Where duplicates come from	Guard	Code
1. Client to API	Users double-click, apps retry after a timeout or a lost reply	`Idempotency-Key` header, `idempotency_keys` table	IdempotentExecutor
2. Broker to consumer	RabbitMQ delivers at-least-once; the relay can publish a row twice	`inbox` table keyed by message ID	NotificationConsumer

One guard is not enough, because each boundary is a different place where a message can repeat. (A third boundary, our service to an outside provider, is described in 6.8 as a design note. We did not build it.)

6.2 Layer 1 step by step: IdempotentExecutor

Plain English: before doing anything, the server writes the client's key into a table. The unique index allows only one writer per key. The winner does the work and files the receipt. Everyone else is handed the receipt.

```
@Transactional
public Stored execute(scope, key, requestHash, Supplier<Stored> effect) {
  int inserted = jdbc.update(INSERT INTO idempotency_keys ... ON CONFLICT DO
NOTHING);
  if (inserted == 0) {                       // somebody already owns the key
    Existing e = loadExisting(scope, key);
    if (!e.requestHash().equals(requestHash))
      throw new KeyReuseException();         // 422
    return new Stored(e.code(), e.body(), true);   // replay
  }
  Stored result = effect.get();              // ledger append + outbox row,
SAME transaction
  jdbc.update(UPDATE idempotency_keys SET status = COMPLETED, ...);
  return result;
}
```

Situation	What happens	Client sees
First time	Insert wins, effect runs, receipt saved	201, Idempotent-Replayed: false
Same key, same payload, later	Insert returns 0 rows, hashes match, saved receipt returned	Same body, Idempotent-Replayed: true
Same key, different payload	Hashes differ	422 IDEMPOTENCY_KEY_REUSED
Same key at the same instant	Second insert waits for the first transaction, then acts like row 2	Idempotent-Replayed: true
First attempt rolled back	Its key row vanished, so the waiting request becomes the owner	Normal success
Key older than 24 hours	Expired row deleted, request treated as new	New effect

Situation	What happens	Client sees
No key at all	Rejected before the executor runs	400 MISSING_IDEMPOTENCY_KEY

6.3 Why the key and the effect must commit together

There are two ways to get this wrong. One transaction closes both:

- **Effect saved, key not saved:** a retry finds no key and repeats the effect. Double credit.
- **Key saved, effect not saved:** a retry finds the key and replays a 'success' for money that never moved. Silent loss.

Because the key row, the ledger event and the outbox row are written in one transaction, they all exist or none do. This is the single most important sentence of the viva.

6.4 Design details examiners ask about

- **Key scope.** The key is stored with a scope like `POST /wallets/{id}/topups`, so the same key on a different endpoint or wallet is a different operation.
- **Request hash.** A SHA-256 of a standard string (scope and amount). It catches a client reusing a key for a different request, which is a bug worth failing loudly on.
- **TTL.** Keys expire after 24 hours and a scheduled job (IdempotencyCleanup) deletes old rows, so the table cannot grow forever. After expiry a very late retry counts as new, so the TTL must be longer than any realistic client retry window.
- **Idempotent-Replayed header.** Tells the client (and the examiners) that this answer came from the saved receipt.
- **Rejected requests are not remembered.** If the effect throws (for example insufficient funds), the transaction rolls back and the key row disappears, so a retry is judged fresh. That is deliberate: the balance may have changed since the first try.

6.5 Why the retry loop sits outside the transaction

`ConflictRetry` wraps the call to `executor.execute(...)`. A transaction that hit a duplicate-key error is already poisoned and must roll back. Each retry needs a brand-new transaction, which only happens if the loop is outside the transactional method. The random delay (jitter, 5 to 25 ms times the attempt number) stops competing requests from retrying in step.

6.6 Walk-through: 50 identical requests at once

1. All 50 try to insert the same key. The unique index lets one through. The other 49 wait.
2. The winner adds one event, updates the balance and commits.
3. Each waiting request wakes up, finds the committed row with a matching hash, and returns the saved receipt.
4. Result: one ledger event, balance +1000, and 49 responses with `Idempotent-Replayed: true`. The counter `wallet.duplicates.suppressed{layer=api}` reads 49.

The e2e test, the k6 script and the Postman collection (folder 3) all check exactly this.

6.7 Layer 2: the inbox

The consumer first inserts the message ID into `inbox` with ON CONFLICT DO NOTHING. If zero rows were inserted, it is a duplicate: acknowledge and skip. The inbox row and the side effect (the `notification_log` row) share one transaction, for the same reason as in 6.3. If the side effect fails, the inbox row rolls back too, so a redelivery is processed properly later instead of being wrongly skipped.

6.8 Design note: a third boundary (not built)

If wallet-service called an outside payment provider, our own retries could charge twice. The fix is the same idea: send our key (for example walletId:Idempotency-Key) as the provider's `Idempotency-Key`, so the provider returns the same charge each time. We describe this as the next step, and we say honestly that we did not build it, because the assignment was one pattern in one service.

6.9 What idempotency does not do

It cannot make an effect outside our database safe unless the receiver also removes duplicates. A real SMS or email is at-least-once. We say this openly instead of claiming exactly-once.

7. Messaging: middleware, outbox, inbox, dead-letter queue

7.1 Message-oriented middleware and RabbitMQ

Middleware is software between applications so they can talk without knowing each other's details. Message-oriented middleware does this with messages that wait in queues. Analogy: a post office. You drop off a letter and leave. It waits safely until the

recipient collects it. That is decoupling in time.

Term	Meaning	Analogy
Producer	Sends messages	The sender
Exchange	Receives messages and routes them by a routing key	The sorting office
Queue	Stores messages until consumed	A pigeonhole
Binding	Rule linking an exchange to a queue	A sorting rule
Consumer	Receives and processes messages	The recipient
Acknowledgement (ack)	'I processed it, you may delete it'	A signature on delivery
Durable	Survives a broker restart	Letters kept in a safe

Our topology (small on purpose):

Exchange	Routing keys	Queue	Used for
wallet.exchange	wallet.credited, wallet.debited	wallet.events.notification	Domain events to the notification consumer
wallet.dlx	wallet.events.notification.dlq	wallet.events.notification.dlq	Messages that could not be processed

7.2 The dual-write problem

A service must change its database and announce it on the broker. These are two separate systems, so you cannot commit both at once.

- **Save, then publish:** crash in between and the world never hears about the change.
- **Publish, then save:** crash in between and the world hears about a change that never happened.

Analogy: posting a letter and updating your diary as two separate acts. A phone call in the middle can leave them out of step.

7.3 The transactional outbox

Idea: write the message into an outbox table in the same database transaction as the data change. Now they are atomic. A separate worker, the relay, later reads pending rows and publishes them.

How our OutboxRelay works

1. Every second it selects up to 50 PENDING rows with `FOR UPDATE SKIP LOCKED`.
2. It publishes each row to RabbitMQ and sets the message ID to the row's `event_id`.
3. It marks the row PUBLISHED.
4. If publishing fails, it adds 1 to `attempts`. After 10 failures the row becomes DEAD so it cannot block the queue.
5. If RabbitMQ is simply unreachable, it stops and leaves rows PENDING without counting an attempt (a broker outage is not the message's fault).

The outbox does not give exactly-once. If the relay crashes after publishing but before marking the row, it publishes again after restart. That is at-least-once, and it is exactly why the receiving side needs an inbox.

7.4 The inbox

The consumer saves each message ID before acting on it. A repeated ID means a duplicate, so it is acknowledged and skipped. In the demo you can switch on `duplicatePublish`, which makes the relay publish everything twice. The log says 'Duplicate message … skipped' and `notification_log` still holds one row per message.

7.5 Failure handling and the dead-letter queue (DLQ)

A DLQ holds messages that could not be processed after several attempts, so one bad message cannot block everything behind it. Analogy: the 'return to sender' shelf at the post office.

- The listener retries a failing message 3 times with growing delays (0.5 s, then 1 s).
- `default-requeue-rejected: false` means that after the last failure the message is not put back.
- The queue has a dead-letter exchange, so the rejected message lands in the `.dlq` queue.

- A **poison message** always fails. The `failConsumer` chaos switch makes every message poison so you can show the DLQ filling up in the RabbitMQ web page. The money in the wallet stays correct, because the API transaction already committed.

Honest limit: nothing replays DLQ messages automatically. A human or a job must inspect them, fix the cause and re-publish. Call this operational follow-up. The DLQ is a side topic for this assignment; spend your viva time on layers 1 and 2.

7.6 Ordering

Outbox rows are published in `id` order and carry the aggregate's `seq`. The consumer does not rely on global order, because each notification is independent.

8. Metrics, chaos, demo and testing

8.1 Metrics

Observability means understanding what a running system is doing from the outside. For this assignment we need only one number: the counter `wallet.duplicates.suppressed`, tagged `layer=api` or `layer=inbox`. It is visible at `/actuator/metrics/wallet.duplicates.suppressed` (Micrometer through Actuator). Every time a duplicate is stopped, it goes up. In the 50-request storm it reads 49 for the api layer. Distributed tracing tools (OpenTelemetry, Jaeger) are outside our scope.

8.2 Chaos engineering

Break the system on purpose, in a controlled way, to prove it survives. Analogy: a fire drill. You do not wait for a real fire to learn whether the exits work.

Switch	Where	What it simulates	What it proves
dropResponse	POST /admin/chaos?dropResponse=true	Server commits, then the answer is lost (HTTP 503)	A retry with the same key is a replay: money moved once
duplicatePublish	POST /admin/chaos?duplicatePublish=true	Relay publishes every message twice	The inbox skips the duplicate; the counter rises

Switch	Where	What it simulates	What it proves
failConsumer	POST /admin/chaos?failConsumer=true	A consumer that always throws	Retries, then messages land in the DLQ

These endpoints are for demos only. A real system would never expose them, and the default RabbitMQ login (guest) is also for local use only. `ChaosState` uses `volatile` fields so a change made by the admin call is seen at once by the request threads.

8.3 The live demo (practise until it takes under 6 minutes)

Show the failure first, then the fix. You can run it with the Postman collection (IdempotentWallet_Viva_Demo, run folders in order with a 500 ms delay) or with requests.http.

#	Scenario	How	What the examiners see
1	Double click	Same top-up twice	One ledger entry; second answer has Idempotent-Replayed: true
2	Lost response (the core story)	dropResponse on, send, off, retry same key	503, but the retry is a replay; money credited once
3	Parallel storm	Postman folder 3, k6 run, or mvn test -De2e=true	Balance exactly 1000; 1 created, 49 replays; counter = 49
4	Wrong key reuse	Same key, different amount	422 IDEMPOTENCY_KEY_REUSED
5	Duplicate broker message	duplicatePublish on	Log 'Duplicate message skipped'; one notification_log row
6	Replay	GET /balance/replay	Rebuilt balance equals the stored balance (consistent: true)

Optional extras if time allows: a poison message (failConsumer) filling the DLQ at localhost:15672, and the overdraft race (8 parallel different keys, no lost update).

Sentence to say aloud: 'We never claim exactly-once delivery. Delivery is at-least-once; the effect happens once.'

8.4 Testing and the Component Testing lecture

Component testing checks a piece in isolation. End-to-end tests check the pieces together. The lecture also covers stubs, drivers, mocks, boundary values, equivalence partitions, decision tables and state-transition testing.

Test	Kind	What it checks
RequestHasherTest	Unit	Same input gives same hash, different input a different one, length is 64
IdempotencyE2ETest: 50 parallel identical top-ups	End-to-end, concurrency	Exactly one ledger event; 49 replays; balance 1000
IdempotencyE2ETest: same key, different amount	End-to-end	422, balance unchanged
IdempotencyE2ETest: withdrawal beyond balance	End-to-end	409
IdempotencyE2ETest: 8 parallel different keys	End-to-end, race	Balance 800 and lastSeq 8: no lost update
k6/duplicates.js	Load	50 virtual users, one key: final balance exactly 1000
Postman collection	Demo with assertions	Every claim above, with green ticks

End-to-end tests run with `-De2e=true` and need the service running. Plain `mvn test` skips them.

8.5 Honest assessment: the weakest part for marks

The lecture asks for isolated component tests with stubs and mocks. Today the project has one real unit test class and a strong set of end-to-end tests. An examiner may notice. These additions are cheap and score well:

Add this test	Technique from the lecture
IdempotentExecutor with a mocked JdbcTemplate: the outcomes in the table in 6.2	Decision table, mocks

Add this test	Technique from the lecture
ConflictRetry: succeeds on attempt 3; throws TooMuchContentionException after the maximum	Boundary values
LedgerService.append: amount equal to balance passes, balance minus one fails, zero amount rejected	Boundary value analysis, equivalence partitions
Random mixed operations never break 'balance = top-ups minus withdrawals'	Invariant (property-style) testing

9. Java and Spring Boot elements

You know Spring Boot basics, so this ties each feature to the reason we use it. The reason is what a viva question usually asks.

9.1 The framework features

Element	Plain meaning	Why it is here
Spring Boot starters, auto-configuration	Add a dependency and Boot configures it	Seven dependencies give a web server, database access, migrations, a broker client and metrics
Dependency injection	Spring builds objects and hands them what they need	Small, replaceable services, so tests can pass in fakes
@RestController, @PostMapping	Turn methods into HTTP endpoints	The public REST API
@RequestHeader	Reads an HTTP header	Reads Idempotency-Key and X-Trace-Id
Bean Validation (@Valid, @Positive)	Declarative input rules	Rejects zero or negative amounts before any logic runs
Java record	Compact immutable data holder	Request bodies and small results (Stored, Row)

Element	Plain meaning	Why it is here
@RestControllerAdvice	One place that maps exceptions to HTTP responses	422 key reuse, 409 no funds, 404 missing wallet
@Transactional	Wrap a method in a database transaction	The heart of the project: key, effect and outbox commit together
Transaction propagation	How a method joins an existing transaction	MANDATORY makes LedgerService.append refuse to run outside one
JdbcTemplate	Plain SQL with little ceremony	We need ON CONFLICT, SKIP LOCKED and exact control
@Scheduled	Run a method on a timer	Outbox relay and key cleanup
Spring AMQP (RabbitTemplate, @RabbitListener)	Send and receive RabbitMQ messages	Publishing outbox rows; consuming events
Listener retry settings	Retry a failing consumer, then reject	Feeds the dead-letter queue
Actuator and Micrometer	Built-in health and metrics endpoints	The duplicates-suppressed counter
Flyway	Runs versioned SQL at startup	Same tables on every machine
Maven	Dependency and build tool	One pom.xml
Docker Compose	Describes several containers in one file	PostgreSQL and RabbitMQ with docker compose up -d

9.2 Java details worth knowing

- **Lambdas and Supplier<T>.** `executor.execute(..., Supplier<Stored> effect)` takes 'the work to do' as a parameter. The executor decides whether and when to run it.

- **volatile fields in ChaosState.** A change by one thread (the admin call) must be visible to other threads (request handlers). `volatile` guarantees that.
- **ThreadLocalRandom** gives jitter without sharing a lock between threads.
- **java.net.http.HttpClient** is the built-in HTTP client, used by the end-to-end tests, so no extra library is needed.
- **jakarta.* packages.** Modern Spring uses `jakarta.validation` and `jakarta.servlet`, not the old `javax.*`.

9.3 Why JdbcTemplate and not JPA/Hibernate

JPA hides SQL behind objects, which is great for ordinary CRUD. Here the SQL is the design: ON CONFLICT DO NOTHING, UNIQUE (wallet_id, seq), FOR UPDATE SKIP LOCKED. With JdbcTemplate every statement is visible, explainable and testable. Cost: more hand-written SQL. Say the trade-off was chosen deliberately. (If `spring-boot-starter-data-jpa` or Lombok are still in your pom.xml, remove them, so the code matches this answer.)

9.4 Spring traps this project avoids (and one it accepts)

1. **@Transactional works through a proxy.** Calling a transactional method from another method in the same class skips the proxy and the transaction. That is why IdempotentExecutor is its own bean.
2. **A poisoned transaction cannot continue.** After a duplicate-key error it must roll back. So retries happen outside the transactional method (ConflictRetry).
3. **Rollback rules.** Spring rolls back on unchecked exceptions by default, not checked ones. Our domain errors extend RuntimeException.
4. **Accepted shortcut.** Some JSON responses and messages are built as strings (key=value&key=value). Easy to read, but production would use typed objects and a schema.

10. Project manager view

A project manager asks five questions: what are we building, who does what, when, what could go wrong, and how do we know it is done. Dates are in weeks because the real submission date is unknown.

10.1 Charter

Item	Statement
Objective	Show a working wallet service whose money operations are safe under retries, duplicates and lost replies, using the Idempotency & Exactly-Once(ish) pattern
Success criteria	All 6 demo scenarios pass live; the 50-parallel-duplicate test shows exactly one effect; every member can explain both idempotency layers in the viva
In scope	Wallets, top-up, withdrawal, event-sourced ledger, idempotent API (layer 1), outbox and inbox (layer 2), chaos switches, tests, Postman demo, report
Out of scope	Saga and second service, resilience tools (breaker, bulkhead, provider calls), distributed tracing, real bank integration, real authentication, a polished UI, cloud deployment
Stakeholders	The lecturer (client and assessor), the viva panel, the group members
Constraints	Uneven Java experience; a fixed deadline; limited AI-assistance usage; one stack (Java, Spring Boot)
Assumptions	Every laptop can run Docker; the README code compiles after small fixes; the viva may be individual

10.2 Roadmap (3 gates)

Week	Phase	Work	Gate at the end
1	Foundations	Agree contracts, repo, Compose, Flyway tables, name role owners	Gate 1: contracts agreed
2	Thin slice	Idempotent top-up, ledger + outbox, relay to RabbitMQ, inbox consumer	Gate 2: slice demo passes

Week	Phase	Work	Gate at the end
3	Prove and polish	Chaos switches, tests, k6, Postman demo, two viva rehearsals, report and slides	Done

A gate is a go/no-go check. If the group does not pass it, next week's work does not start. After the feature freeze only tests, fixes, the report and rehearsals are allowed. A smaller system that survives every demo beats a bigger one that only works on the happy path. That is exactly why the saga and resilience parts were cut.

10.3 Work breakdown and ownership

#	Role	Deliverable	Done when
1	Idempotency lead	idempotency/*, WalletController, ApiExceptionHandler	The table in 6.2 passes as tests
2	Ledger lead	LedgerService, V1__init.sql, replay endpoint	Overdraft race test passes; replay says consistent
3	Messaging lead	RabbitConfig, OutboxRelay, NotificationConsumer	Duplicate delivery is skipped; poison message reaches the DLQ
4	Chaos and load lead	Chaos switches, k6, Postman collection, demo runbook	Demo runs end to end in under 6 minutes, twice
5	Quality lead	Unit and component tests	The extra tests in 8.5 exist and pass
6	Docs and slides lead	Report, slides, screenshots	Report matches the code; every claim has a screenshot

Each owner is also the viva expert for their part, but everyone must be able to explain the whole flow.

10.4 Dependencies and critical path

The longest chain that cannot be shortened: agreed contracts -> ledger -> idempotent executor -> outbox and relay -> chaos demo. If the ledger slips, almost everything slips, so the ledger lead should be your strongest Java developer. Work that does not block the chain: k6 scripts, the Postman collection, the report skeleton, and tests written against the agreed contracts.

10.5 Risk register

Risk	Likelihood	Impact	Mitigation	Owner
README code has compile errors (never compiled by its author)	High	Medium	Fix on day one with one person; share the fixed commit	1
Flyway checksum mismatch after editing V1	Medium	Medium	Never edit an applied migration; in dev use docker compose down -v	2
Uneven Java skill leaves one person carrying the work	High	High	Pair weak members with role owners; rotate who runs the demo	all
Individual viva exposes members who built only one part	High	High	Two rehearsals where each member explains a part they did not build	5
Docker or port problems on some laptops	Medium	Medium	Shared setup-check; a fallback machine for the demo	4
End-to-end tests are timing-sensitive and flake	Medium	Medium	Use the polling helper; no fixed sleeps; run twice before demo day	5
Scope creep (saga, resilience, UI)	Medium	High	Scope rule: only Idempotency & Exactly-Once(ish); a 'parking lot' list for ideas	PM
Demo fails live	Medium	High	Rehearse twice; keep a recorded backup of every scenario	4

Risk	Likelihood	Impact	Mitigation	Owner
Attribution issue with the reference repository	Low	High	State in the report that the idea was inspired by it and the code is a new Java design	6

10.6 Definition of done

- The service starts with `docker compose up -d` and one `mvn` run, with no manual database edits.
- All 6 demo scenarios pass on a clean machine (Postman collection all green).
- `mvn test` passes; `mvn test -De2e=true` passes with the service running.
- The extra component tests from 8.5 exist.
- The report matches the code (names, ports, tables) and includes diagrams and screenshots.
- Every member has explained both idempotency layers once, out loud, to another member.
- The limitations in section 11 are written into the report.

10.7 What examiners probably reward

This is an informed guess from the slides. Check it against the real assessment brief.

Likely criterion	Evidence we show
Understanding of Idempotency and Exactly-Once(ish)	Two layers, the same-transaction rule, honest wording about 'ish'
Use of the assigned design pattern	Idempotency key, request hash, replay, outbox, inbox
Working system and demo	6 live scenarios, failure first then the fix
Testing quality	Unit, end-to-end, load tests and the Postman assertions
Design justification	Why optimistic concurrency, why JdbcTemplate, why one transaction
Teamwork and individual understanding	Roles, Git history, viva rehearsals

10.8 Ways of working

- **Cadence:** two 15-minute stand-ups a week (done, next, blocked), plus the weekly gate review.
- **Git:** one repository, protected main, a feature/<role> branch each, merge by pull request with one reviewer. Do not commit `target/`.
- **Decision log:** a short file listing each design decision, the options and the reason. It becomes your report's justification section almost for free. Our big decision: cut the saga and resilience to focus on the assigned pattern.
- **Measures:** demo scenarios passing (target 6 of 6), tests passing, duplicates suppressed in the 50-parallel run (target 49).

11. Viva playbook, limitations, glossary, study plan

11.1 Strategy

1. **Lead with the problem, not the technology.** Panels remember the lost-reply story better than a list of patterns.
2. **Show failure first, then the fix.** Show the 503, then the retry that returns a replay with no second credit.
3. **Say 'effectively-once', never 'exactly-once'.** Explain why 'ish' is honest.

4. **Name your own limits before they do.** It reads as maturity, not weakness.
5. **Answer in your own words.** If you do not know, say how you would find out. Do not bluff.
6. **Point at code.** For each answer, open the one file that proves it.

60-second pitch: 'Our system is a digital wallet. The problem is that networks lose replies, so a customer's retry can move money twice. We solved it by giving every request an idempotency key and saving the key, the ledger event and the outgoing message in a single database transaction. Messages travel through an outbox to RabbitMQ and are de-duplicated by an inbox on the other side. To prove it, we break the system on purpose: we drop responses, duplicate messages and flood it with fifty identical requests, and the money always adds up.'

11.2 Viva questions and strong answers

Question	Strong answer
Is exactly-once possible?	Not over an unreliable network: a sender cannot tell a lost request from a lost reply. We combine at-least-once delivery with idempotent handlers, so the effect happens once ('ish').
Isn't this just a unique constraint?	A unique constraint stops the duplicate but gives the client an error. We also store and replay the original response, check the payload hash, and make the effect and the record atomic.
Who generates the key?	The client, once per intention (a UUID per 'Send' tap), reused on every retry. A server-generated key would differ each attempt and defeat the purpose.
Why must key and effect commit together?	Effect without key: a retry repeats it. Key without effect: the client is told 'success' for money that never moved. One transaction closes both gaps.
Same key, different payload?	Compare request_hash and answer 422. Silently replaying would hide a client bug.

Question	Strong answer
Two identical requests at once?	The unique index lets one insert win; the other waits, then replays the committed result.
What if the idempotency table is lost?	Replays become new operations. That is why the ledger has UNIQUE (wallet_id, seq): defence in depth. In production the table lives in the same durable database as the ledger.
What if a client uses a weak or guessable key?	Keys are scoped to endpoint and wallet, and a reused key with a different payload is rejected. In production, add the user ID to the scope.
What happens at 24 hours?	The row is deleted and a very late retry is treated as new. The TTL must exceed any realistic client retry window.
Why not store keys in Redis?	Redis is a different system from the ledger, so the key and the effect could not commit atomically. That would bring back the exact gap we closed.
What is a dual write? Why an outbox?	Writing to the database and the broker separately can disagree after a crash. The outbox row commits with the data; a relay publishes it later.
Does the outbox give exactly-once?	No. A relay crash after publishing and before marking gives a duplicate. That is why the receiver needs an inbox.
Is outbox plus inbox overkill?	Each solves a different failure: the outbox stops a lost message on publish; the inbox stops a double effect on delivery. Remove either and a failure case comes back.
Why is the retry loop outside the transaction?	After a duplicate-key error the transaction is poisoned; each retry needs a fresh one. Jitter stops retries moving in lockstep.
Why optimistic concurrency?	No locks held, so no deadlock; a clash is detected by WHERE last_seq = ? and retried.

Question	Strong answer
Why RabbitMQ and not Kafka?	We need task-style queues, routing keys, acknowledgements and dead-letter queues, which RabbitMQ gives out of the box. Kafka suits replayable high-volume event logs and would be heavier than needed.
What if two relays publish the same row?	SKIP LOCKED prevents it in normal running. If it ever happened, the inbox dedupes it.
How would it scale?	Ledger: partition wallets across instances (one writer per wallet). Relay: SKIP LOCKED with several instances. Consumers: competing consumers on the queue.
How do you know the money is safe?	The invariant: balance equals top-ups minus withdrawals, checked after every demo scenario (the Postman collection asserts it) and by replaying the ledger.
Why did you not build a saga or resilience?	The lecturer assigned one pattern: Idempotency & Exactly-Once(ish). We kept the scope to what that pattern needs and made it solid.
What does it NOT guarantee?	Effects outside our database (an email, a real SMS) are at-least-once unless the receiver dedupes. We say so instead of claiming exactly-once.

11.3 Honest limitations (say these first)

Limitation	Effect	How to phrase it	Fix
No publisher confirms on the relay	A broker crash at the wrong moment could lose one message	'Delivery is at-least-once from the relay, but we do not yet use confirms.'	Enable publisher confirms; mark PUBLISHED only on confirm
Downstream idempotency not built	Calls to an outside provider would not be protected	'We describe it in 6.8 but the assignment was one service.'	Send our key as the provider's Idempotency-Key

Limitation	Effect	How to phrase it	Fix
Balance projection updated synchronously	Always consistent, so simpler than true CQRS	'A deliberate simplification.'	Update it asynchronously from the outbox stream
Keys not tied to a user	No authentication exists	'Out of scope.'	Add the user ID to the key scope
Messages are key=value strings	Fragile and untyped	'Chosen to avoid libraries.'	JSON or Avro with a schema
Chaos endpoints open; RabbitMQ default login	Insecure outside a demo	'Demo only.'	Remove or protect them
No automatic replay of dead-letter messages	A human must act	'Operational follow-up.'	A small admin job to inspect and republish
No snapshots	Replay slows on very long histories	'Not needed at this scale.'	Snapshot every N events
Component tests are thin	Weaker marks on the testing lecture	Fix it, do not explain it	The additions in 8.5

11.4 Glossary

Term	Plain meaning
Idempotent	Doing it many times has the same effect as doing it once
Idempotency key	A client-chosen ID that names one intention, reused on every retry
Effectively-once	At-least-once delivery plus idempotent handling, so the effect happens once
Replay (API)	Answering a repeat request from the saved receipt
Replay (ledger)	Rebuilding state by re-applying events
Event sourcing	Store the history of facts, work out the state from it

Term	Plain meaning
CQRS	Separate model for writing and for reading
Projection	A read model built from events
Optimistic concurrency	Detect a clash when saving instead of locking first
Transaction	A group of changes that succeed or fail together
Outbox	A table of messages committed with the data, published later
Inbox	A table of processed message IDs used to skip duplicates
Relay	The worker that moves outbox rows to the broker
Dead-letter queue	A holding queue for messages that keep failing
Poison message	A message that always fails when processed
Jitter	Random variation in retry delay
Flyway checksum	A fingerprint of an applied migration; if the file changes, startup fails
Chaos engineering	Breaking things on purpose to prove resilience

11.5 Study plan

Five sessions of about 45 minutes. Each ends with a whiteboard drill: explain out loud with no notes.

Session	Read	Whiteboard drill
1	Sections 1 and 2	Draw the lost-reply timeline and explain why a retry is unavoidable
2	Sections 4 and 5	Draw the overdraft race and how the unique constraint resolves it
3	Section 6	Draw the two layers and walk through 50 parallel identical requests

Session	Read	Whiteboard drill
4	Section 7	Draw outbox -> relay -> broker -> inbox, and say what happens if the relay crashes
5	Sections 8 and 11	Run the Postman demo once, then answer ten random questions from 11.2

When you can pass every drill without looking, you are ready. A good test: explain one layer you did not build to a teammate, who then tries to find a hole in it.